Check for updates

d from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwrXcJAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgkQp2lIPRKnjoFGTr58PMjg by Universita Di Pisa user on 27 November 2

Leveraging Task Modularity in Reinforcement Learning for Adaptable Industry 4.0 Automation

Qiliang Chen
Department of Mechanical and Industrial Engineering,
Northeastern University,
Boston, MA 02115
e-mail: chen.qil@northeastern.edu

Babak Heydari
Department of Mechanical and Industrial Engineering,
Northeastern University,
Boston, MA 02115
e-mail: b.heydari@northeastern.edu

Mohsen Moghaddam[1]
Department of Mechanical and Industrial Engineering,
Northeastern University,
Boston, MA 02115
e-mail: mohsen@northeastern.edu

The vision of Industry 4.0 is to materialize the notion of a lot-size of one through enhanced adaptability and resilience of manufacturing and logistics operations to dynamic changes or deviations on the shop floor. This article is motivated by the lack of formal methods for efficient transfer of knowledge across different yet interrelated tasks, with special reference to collaborative robotic operations such as material handling, machine tending, assembly, and inspection. We propose a meta reinforcement learning framework to enhance the adaptability of collaborative robots to new tasks through task modularization and efficient transfer of policies from previously learned task modules. Our experiments on the OpenAI Gym Robotics environments Reach, Push, and Pick-and-Place indicate an average 75% reduction in the number of iterations to achieve a 60% success rate as well as a 50%-80% improvement in task completion efficiency, compared to the deep deterministic policy gradient (DDPG) algorithm as a baseline. The significant improvements achieved in the jumpstart and asymptotic performance of the robot create new opportunities for investigating the current limitations of learning robots in industrial settings, associated with sample inefficiency and specialization on one task through modularization and transfer learning. [DOI: 10.1115/1.4049531]

Keywords: Industry 4.0, collaborative robots, meta-learning, combinatorial generalization, reinforcement learning, transfer learning

1 Introduction

The evolution of industries has always been driven by new technologies that facilitate improvements such as increased throughput, lower costs, higher quality, and augmented human performance. Industry 4.0, the fourth industrial revolution, is characterized by emerging technologies that fuse the physical and digital dimensions of manufacturing environments, driven by breakthroughs in a multitude of areas such as Internet-of-things, automation and robotics, human-machine interaction, and artificial intelligence (AI) [1–4]. The vision is to realize the notion of *lot-size of one* by creating adaptive, resilient, and collaborative factory systems [5] that enable low-volume, high-variability production with minimal downtime associated with reprogramming or reconfiguration of machines [6–9]. Industrial robots are key enablers for this vision due to their versatility and extensive use in a broad spectrum of manufacturing operations including part transfer, welding, painting, packaging, labeling, assembly, and inspection, among others. Traditional industrial robots, however, are suited for operating in structured environments and on rote, repetitive tasks. Although this appears to be an expedient feature for operations such as welding and painting with minimal likelihood of variation in process parameters, operations such as part transfer, assembly, and inspection demand certain degrees of adaptability and resilience, especially when performed by *collaborative robots* and in close proximity with humans. We develop and test a computational framework built upon recent advances in deep reinforcement learning (RL) and transfer learning, combined with the notion of modularity, to enhance the adaptability and resilience of collaborative robotics operations in Industry 4.0 environments.

1.1 Collaborative Robots in Industry 4.0.

Collaborative robots, also known as *cobots*, are relatively small, "cage-free" robotic arms with built-in safety features such as force feedback and collision detection that allow them to function in close proximity with humans. With the vision of developing intelligent and flexible robotic arms that could serve as a "tool" for factory workers [10], the global market for cobots is anticipated to reach $10.14 billion by 2025 [11]. Cobots are intended to complement and augment the performance of factory workers in operations that are manual and tiresome, and yet hard to fully automate due to the inherent variations in process parameters. Major Industry 4.0 applications of cobots include (Figure 1):

- *Material handling*. Cobots have been widely adopted in recent years for performing part transfer, box assembly, packing, collating, and palletizing tasks alongside humans, with high speed, accuracy, and repeatability, as well as reduced cost and downtime for changeovers. Current material handling applications of cobots primarily involve repetitive tasks in medium-/high-volume production environments.
- *Machine tending*. Cobots can assist machinists in loading/unloading parts to/from a variety of process equipment such as CNC milling and turning machines and injection molding machines, as well as welding, stamping, forging, or punching stations. Current machine tending cobots are typically used to carry out repetitive tasks that are tiresome, hazardous, and/or prone to error for humans.
- *Assembly*. Cobots can work alongside manual assembly workers to facilitate their speed and accuracy on complex assembly tasks. Applications of cobots for product assembly include feeding the parts into the assembly workstation,

[1]Corresponding author.

Contributed by the Design Automation Committee of ASME for publication in the Journal of Mechanical Design. Manuscript received May 26, 2020; final manuscript received December 14, 2020; published online January 29, 2021. Assoc. Editor: Jitesh H. Panchal.

 Copyright © 2021 by ASME

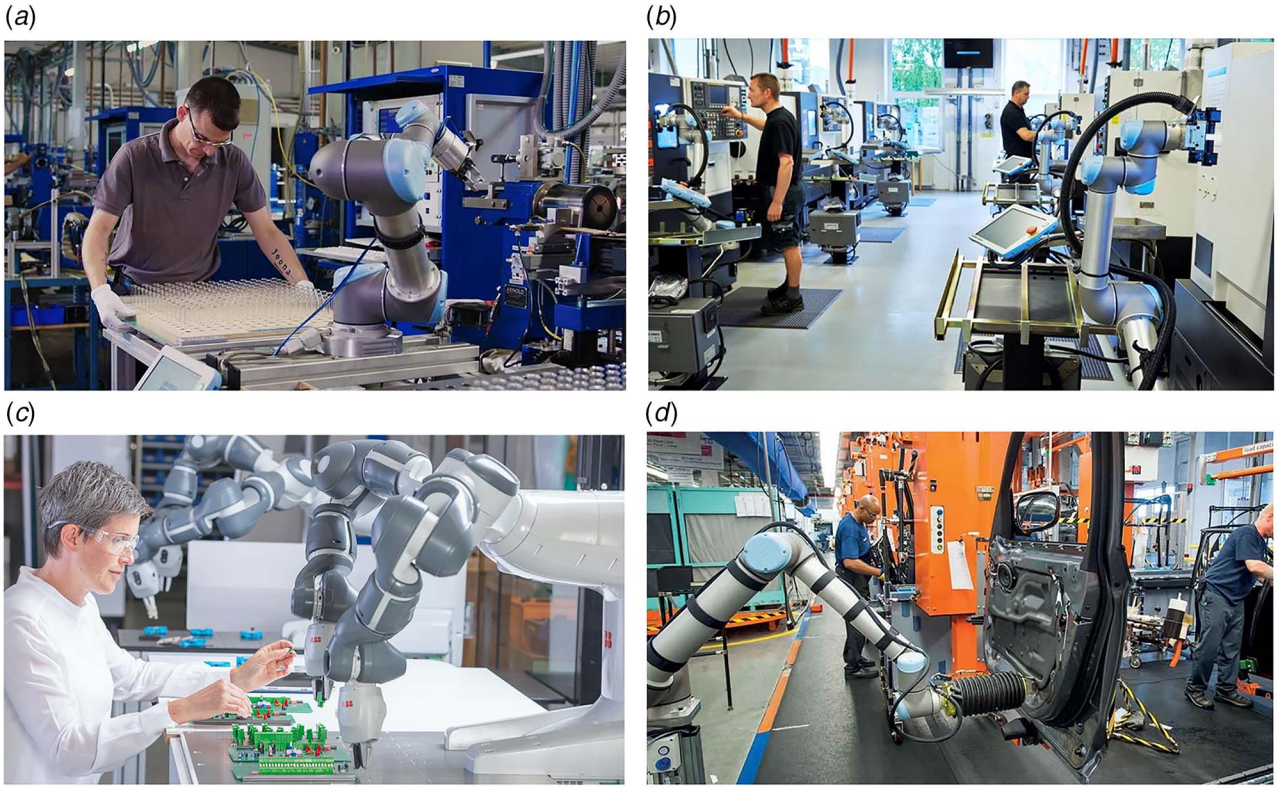


Fig. 1 Examples of cobot applications in manufacturing: (*a*) material handling: pick-and-insertion of parts into a bin (courtesy of Universal Robots), (*b*) machine tending: loading/unloading parts into/from CNC machines (courtesy of BRAAS Robotics), (*c*) collaborative assembly in tandem with a human worker (Courtesy of ABB), and (*d*) inspection alongside humans in the automotive industry (courtesy of Universal Robots)

aligning, adjusting, or mounting the parts on the product platform, and fastening the components [12]. These tasks are accomplished in collaboration with assembly workers and possibly other cobots.

- *Inspection*. Manual inspection, testing, or measurement of products is a dull, tedious task that is highly prone to errors. Cobots can facilitate this complex process by enabling rapid, accurate, and repeatable visual verification of machined parts or assembled products against their CAD models using high-resolution cameras.

Current research on the integration of cobots into factories is predominantly focused on stable and safe operations rather than on dynamically changing environments and applications [13]. That is, in spite of their inherent versatility and flexibility, there is no guarantee that cobots can efficiently *adapt* to potential variations in the size, geometries, or positions of the parts to be handled, inspected, or loaded into a process equipment, or in the sequence or process requirements of a multi-product assembly cell. To bridge this gap, the versatile and flexible hardware structure of cobots must be enhanced with advanced computational techniques and simulation tools to fully materialize the degrees of adaptability and resilience required to cope with the dynamic changes on the shop floor.

1.2 Enabling Adaptable and Resilient Industry 4.0 Cobots.

Adaptability is an essential characteristic of systems that face a high degree of uncertainty and variable requirements and has been mentioned as one of the key features of Industry 4.0 systems as a response to a combination of high product variability and short product life-cycle [14,15]. Such systems need to have a high degree of resilience in response to unanticipated disruptions in the supply chain, market demands, changes in regulations, cyber-attacks, and rapid technological changes [16,17]. Although traditional automation cannot deliver such a level of adaptability and resilience, recent breakthroughs in AI, specifically in deep RL and machine vision, have created new possibilities to enhance the adaptability and resilience of cobots in dynamically changing Industry 4.0 environments. RL has recently been applied in a variety of design applications such as in control of shared energy assets [18], operation of distributed energy resources [19], global routing problems [20], and design optimization [21] (see also the 2019 JMD special issue on machine learning for engineering design [22]). However, standard RL by itself cannot necessarily improve adaptability and resilience as it often requires reprogramming and relearning (almost from scratch) when facing a new task, hampering its efficient implementation. For example, a cobot can only insert a particular set of chips on a board or move objects of similar shape and size from/to fixed locations in a warehouse. This knowledge gap, which has turned into a core subject of study by many researchers and institutions such as DeepMind [23–27], Siemens, and OpenAI [28–31], is due to *the lack of transfer between tasks*, which, in turn, hinders the adaptability of robotic operations to the dynamic variations in demand or unexpected variations on the shop floor.

Meta-learning [32] is an emerging topic in AI, which aims at leveraging past experience to accelerate adapting to a new task. The motivation stems from two fundamental limitations of current deep learning methods [23,27]: (1) sample inefficiency (i.e., relying on a large number of experience episodes) and (2) single-task specialization. Deep RL, for instance, has recently achieved remarkable success in learning complex tasks such as playing Atari games [33], Go [34], and poker [35]; however, these RL algorithms take several days or weeks to train, and once trained, can only handle one task. Meta RL tackles this limitation by training the agent not only on one task but on a series of interrelated tasks drawn from a given distribution or a prior [23,27,29,36]. Meta RL has recently been enhanced with an episodic memory which

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwxXcJAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgIcQp2IIPRKnjoFGT58PMjg by Universita Di Pisa user on 27 November 2

allows the agent to better estimate the value of a state and action by reinstating memories of similar past states and actions [26,37,38]. Section 2 provides an extensive review of meta-learning and transfer learning methods in the context of deep RL.

In spite of significant progress in addressing the first limitation of conventional deep RL (i.e., sample inefficiency) through meta RL, the second limitation (i.e., specialization on one task) remains a challenging, unsolved problem. Current meta RL methods are still limited to very narrow task distributions that allow slight parametric differences between individual tasks [31]. That is far away from the generalization ability needed in real-world Industry 4.0 settings, and certainly at a considerable distance to the level of adaptability that human agents are capable of. Building on the existing work in meta RL, we take a high-level combinatorial generalization approach based on the notion of *modularity* [39,40] to address the second limitation of deep RL. As argued by Herbert Simon [41], modularity is the ubiquitous adaptability mechanism through which most biological, social, economic, and software systems manage complexity in highly unstructured environments [42–46]. The implication of modular design in the context of deep RL is simple: Although learning a single policy that performs optimally across all tasks is non-optimal [39], learning and mixing-and-matching simpler policies for sufficiently small and overlapping task modules may solve the second problem of current deep RL methods concerned with specialization on one task. By doing so, we aim at gaining agent adaptability by accepting some extra cost and complexity that comes with modular architectures, whose level depend on the diversity and uncertainty of anticipated tasks.

1.3 Objectives and Outline. We propose and test a meta RL framework based on the notions of modularity and transfer learning to enable generalization to diverse collaborative robotic operations in Industry 4.0. Our goal is to tackle the following fundamental research question: *How can a cobot use task modularity to autonomously adapt to complex and unstructured Industry 4.0 operations and environments through the automated transfer of its learned knowledge across several tasks with parametric and/or non-parametric variations?* We first decompose tasks into reusable modules for combinatorial generalization to a broad range of new tasks. The underlying assumption of task modularization is that there exists a *composition function* [40] that enables the formation of any new task from a set of task modules. Building on modular task representations, we then train a separate deep neural network for undertaking each individual task module. The idea is that these module neural networks can be reused to boost the training of the agent on new parametric and non-parametric variations of the learned tasks in the future [40]. The initialization is performed by transferring the neural network parameters of the most similar pre-learned module—if no such similar pre-learned module exists, the parameters are initialized randomly. This transfer learning approach for RL [47] is similar to the notion of episodic RL which transfers action/state values from the episodic memory as the agent goes through the learning process [26].

Following the current standard practice in RL research, the proposed framework is validated in the Robotics environments of the OpenAI Gym [48]. We use deep deterministic policy gradient (DDPG) [49] as our baseline for the deep RL model. Simulation toolkits such as OpenAI Gym are instrumental for developing, testing, and validating deep RL algorithms that typically require hundreds of thousands of "trial-and-error" iterations before learning a certain task. Direct execution of such learning processes on physical shop floor settings is costly and unjustifiable during the initial research stages. The Robotics environment of OpenAI Gym simulates four robotic tasks including reach, push, slide, and pick-and-place, which despite being "simplistic" versions of real-world tasks, provide a solid platform for testing and validation of proof-of-concept models and algorithms.

The remainder of this article is organized as follows. Section 2 presents an overview of deep RL and meta RL literature for continuous control of robotic arms. Section 3 further elaborates on the notions of modularity and meta-learning in the context of the presented work. Section 4 provides the details of the proposed meta-RL framework. Section 5 presents the experiments, results, and discussions, and Sec. 6 concludes the article and provides a summary of limitations and directions for future research.

2 Background and Related Work

This section presents a brief yet comprehensive review of the background and related work on continuous space RL and meta-learning. Readers familiar with these topics may skip this section without losing the scope of the work.

2.1 Continuous-Space Reinforcement Learning. RL [50] is the process of learning by doing, where an agent gets better at interacting with its environment by continually taking actions and receiving rewards. Although the theory of RL and some of its fundamental algorithms were developed decades ago, their practical use had been limited largely due to the impracticality of scaling RL to real-world problems with large state and action spaces. This so-called *curse of dimensionality* [51] was significantly mitigated through the efficient use of deep neural networks for approximating different value functions. Deep RL achieved incredible success just a few years ago by reaching super-human performance in playing Atari games [33]. This inspiring result was followed by a series of more challenging tasks such as playing the game of *Go*–the holy grail of AI for decades: In 2016, the deep RL agent AlphaGo developed by DeepMind defeated the legendary Go player Lee Sedol 4-1 using deep RL algorithm [52], and only a year later, the updated agent AlphaGo Zero achieved superhuman performance, winning 100–0 against the champion-defeating AlphaGo [34]. StarCraft II is a real-time strategy video game with a highly complex environment and diverse decision options. In 2019, AlphaStar developed by DeepMind using multi-agent RL mastered this game and was rated above 99.8% of officially ranked human players [53]. In a more recent work, Ref. [54] developed a safe RL algorithm to teach a humanoid robot to walk faster.

a Value-Based versus Policy-Based Methods. RL algorithms can be model-free or model-based. The difference is that unlike model-free RL, model-based RL must first learn a representation of the environment and plan a solution accordingly (e.g., Refs. [55,56]). Q-learning [50] is a popular and powerful instance of value-based, model-free RL. Deep Q-network (DQN) estimates the Q-value function with deep neural networks, which serves as an indicator of the goodness of taking an action given a specific state, guiding the policy in choosing the actions that maximize the Q-value [33]. In addition to *value-based* methods, there is another family of *policy-based* RL methods, which update the policy directly based on the gradient calculated from the reward parameters [57]. This feature is particularly suited for robotic agents with continuous action spaces. The value-based methods are typically referred to as 'critic-only," while the policy-based methods are known as 'actor-only." The combination of these two, which serves as our baseline model, is actor-critic RL, as described next.

b Actor-Critic Methods. The actor-critic architecture leverages the strengths of both value-based RL (i.e., learning from off-policy experiences) and policy-based RL (i.e., directly optimizing actions). In actor-critic RL, the critic approximates a value function which is then used to update the actor's policy parameters for performance improvements [58]. One of the biggest drawbacks of the original actor-critic RL method is high variance, which has been addressed by the advantage function, which is calculated by subtracting a baseline value from the value function [59]. To make efficient use of the data collected from the interactions with the environment, importance sampling has been proposed to reuse past experiences

nloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=BZN-hBwXcJIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgkQp2IIPRKnjoFGT58PMjg by Universita Di Pisa user on 27 November 2

for updating the policy [60]. The proximal policy optimization (PPO) algorithm adds a constraint on the difference between the old policy and the new policy through importance sampling to learn from previous experiences, which leads to a more stable learning process [61]. As for the problem settings in the mentioned algorithms, the action space is always discrete. In Atari games, for example, the actions may include "up," "down," "left," "right," or "fire." The policy output is therefore a probability distribution of actions given a state, where the optimal policy chooses the action with the highest probability. In practical robotic applications, however, the action space may be *continuous*; e.g, setting the angle or velocity of the robot's joints [62].

Deterministic policy gradient (DPG) uses an actor to map states to deterministic actions, rather than a probability distribution [63]. Thus, it can be applied to continuous action space tasks which are of our interest. Deep deterministic policy gradient (DDPG) is trained off-policy with samples from a replay buffer [33,64], which also applies a target to provide consistent targets during temporal difference backups [49]. DDPG has recently been used in the design literature for energy systems [18]. In the Industry 4.0 collaborative robotic tasks mentioned in Sec. 1, the reward signals are sparse and binary, which means that the cobot may keep interacting with the environment without any positive reward, thus learning nothing. One of the popular methods for solving this problem is *reward shaping*. Building on the prior knowledge of a human, the cobot can be guided to reach the final goal step by step [65]. We build on the hindsight experience replay (HER) for this purpose, which is designed for learning from failure through creating dummy intermediate goals for the cobot [66]. Therefore, even if the cobot does not reach the real goal, it can still receive some reward to learn something from every single experience. We thus utilize DDPG-HER as a baseline for evaluating the proposed modular meta RL framework described in Sec. 4.

2.2 Meta-Learning. Although deep RL has achieved a superhuman level performance in learning single tasks, it still has two major limitations associated with learning efficiency and transferability across different tasks [23], which may hinder the ability of the cobots trained through deep RL to keep up with their human counterparts. First, deep RL is sample-inefficient, while humans can attain reasonable performance on a broad range of tasks with comparatively little experience. Second, deep RL can only specialize in a narrow range of tasks, while humans can flexibly adapt to different variations of the same tasks. These limitations of current deep RL algorithms are known to be the result of weak inductive bias and incremental parameter adjustment [27]. Meta-learning is an emerging concept in deep learning inspired by Harlow's notion of *learning to learn* in psychology [67]. Humans have the ability to accumulate knowledge of past experiences in a special form of memory for value-category matching [68] and utilize that as a consciousness prior to rapidly learn new tasks and adapt to new environments [69]. Meta RL aims at enabling the learning agent to leverage its prior knowledge (i.e., inductive bias) and learn to master new tasks more efficiently. This idea has been investigated from different angles. One approach is to apply recurrent neural networks to train the agent on a series of parametrically different tasks from the same distribution to maximize the total reward on the entire set of tasks [23,29]. The underlying idea is that since the agent takes all the previous information into account, it can learn common knowledge across the tasks, thus accelerating its learning on a new task.

A simple yet powerful meta RL algorithm is model-agnostic meta-learning (MAML), which finds a good initial set of parameters by training on a series of interrelated tasks with the objective of achieving the best performance after a few updates [36]. Mishra et al. [70] combined temporal convolution layers with causal attention layer to avoid the exponentially increasing number of layers when using recurrent neural networks as meta-learner. Gupta et al. [25] introduce a latent state to the inputs of the policy drawn from Gaussian distribution and use MAML to learn a good latent state and improve the exploration process. Experiments on 50 robotics environments by Yu et al. [31] setup a novel benchmark for multi-task meta RL research in the future. Episodic RL is a different approach to meta RL, based on recording the states that the agent encountered in the past and the actions taken, such that when the agent faces a new situation, it can go back to the memory, retrieve the most similar situation, and take action accordingly [38]. Doing so, the agent can identify the best behavior faster without spending too much time learning the parameters from scratch. Ritter et al. [26] combine episodic memory with meta RL architecture, which is shown to yield better performance than previous meta RL models without episodic memory on several experiments.

Both meta RL and episodic RL build on the notion of *transfer learning* [47]. Transfer learning is built upon the notion of transferring the knowledge of previous tasks to a new task such that the agent can master the new task faster. Our proposed framework contributes to this domain by introducing a task modularization approach based on two rationales: (1) related tasks with non-parametric differences can be decomposed into modules that overlap across the tasks and (2) the knowledge of a previously learned task module related to Task A can be reused for initializing the learning of a similar task module associated with Task B. Transfer learning for RL offers several metrics for evaluating the success of such cross-task transfer of knowledge, some of which we utilize for evaluating the performance of the proposed modular DDPG model against the baseline DDPG [47]: jumpstart, asymptotic performance, total reward, transfer ratio, and time-to-completion.

3 Modularity and Meta-Learning

Modularity has long been recognized as an effective adaptability mechanism and has been an active area of research in a wide range of academic disciplines, since the influential work by Herbert Simon [71]. Simon argues that near-decomposability enables systems to respond effectively to external changes without disrupting the system as a whole. Modularity has also been recognized as an essential concept in architecting engineering products, processes, and organizations. It has been shown to increase product and organizational variety [72], the rate of technological and social innovation [73], market dominance through interface capture [74], and cooperation and trust in networked systems [45,75].

The ubiquity of modularity in various complex systems as an adaptability mechanism makes it a suitable candidate to be integrated with RL algorithms to achieve adaptability through efficient transfer learning across different tasks with both parametric and non-parametric variations. Leveraging modularity to scale up RL capabilities dates back to the early 1990s [76,77], following and enabled by the development of the Q-learning algorithm. However, modularity has been used with different meanings and approaches in AI since then and it is important to distinguish between different modes that modularity has been used in meta-learning. With some simplifications, previous works in modular meta-learning can be divided into two general categories based on their overall approach towards modularity. Here, we briefly discuss these two approaches.

The first approach, dominant in the pre-deep RL era, is based on hierarchical learning [78], in which most frameworks consist of two separate mechanisms for task decomposition [79] followed by behavior coordination [77]. In this approach, different modules act as different action voters. That is, each module observes the action taken by the agent, the state transition, and a reward signal specific to the module. At each time-step, the agent combines the action preferences of the modules to compute a joint policy [80–82]. More recently, Frans et al. [83] applied hierarchical RL using deep neural networks by pre-training a pool of common neural network sub-policies across tasks and using a task-specific master-policy neural network to select the appropriate sub-policy.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=BZN-hBwXcJIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgIcQp2IIPRKnjoFGT58PMJg by Universita Di Pisa user on 27 November 2

Unlike the first approach which considers modules as distinct (goal-specific) policy agents whose action recommendations need to be aggregated for each task and the second approach directly applies modularity to the deep neural network architecture. Modules are thus reusable neural network functions that are pre-trained and can be recombined in different ways to undertake new tasks. Here, the basic idea is that rather than training a single network on a large number of training data, one can simultaneously train a large number of different networks, while tying their parameters together, which in the end generates a set of reusable neural network modules. This idea has recently been applied to a variety of applications such as real-world reasoning problems [84], task and motion planning [85], and robotics [86]. In the latter, the authors show that neural network policies can be decomposed into task-specific modules, shared across robots and robot-specific modules, shared across tasks. Our work follows this approach and builds on Ref. [40] in which the authors use a set of neural network modules for a set of basic functions that can be re-tuned and recombined using adaptive structures in the face of new tasks. Unlike Alet et al. [40], who formulate and apply this method to a set of supervised robotic problems, we take this approach and customize it for RL applications.

4 Framework

To achieve the degrees of adaptability and resilience required from collaborative robotic operations in the context of Industry 4.0 (e.g., material handling, assembly, inspection), it is essential to build mechanisms that not only allow the cobots to automatically learn new tasks, but learn them efficiently by *transferring* optimal policies across different tasks. Several RL algorithms have been introduced in recent years for dealing with continuous, high-dimensional action spaces among which the DDPG algorithm [49] has demonstrated great success in accomplishing relatively simple tasks such as pendulum, cartpole swing up, or puck shooting. For more complex, continuous and high-dimensional action space tasks such as the ones performed by cobots, however, current off-policy RL algorithms such as DDPG may not be directly applicable (see, e.g., [66,87]) due to their inherent complexity. These algorithms often take several hours/days to converge and their optimal policies can only be used for the particular tasks learned. We tackle this problem through task modularization and module composition to enable the transfer of the learned policies across different tasks with non-parametric variations (Fig. 2). Without loss of generality, we implement this notion through a modular DDPG (M-DDPG) algorithm and test it on three OpenAI Gym Robotics environments. Details of the proposed framework are presented next (see Algorithm 1).

4.1 Task Modularization. Specifying the degree and architecture of modularity is one of the key decisions to make, governed by a range of parameters that characterize the degree of spatial variations (e.g., distance between different tasks or environments) as well as temporal uncertainties (e.g., unexpected obstacles on the way, or dynamic environments) [88]. In this context, the objective is to decompose tasks into reusable modules for combinatorial generalization to a broad range of new tasks. We present a formalism of the modularization process in this section and leave the development of a methodology for automated identification of the task module for future research. The underlying assumption of task modularization is that there exists a *composition function* [40] for forming any new task from a set of modules. The initialization of the respective modules is thus performed by transferring the neural network parameters of the most similar pre-learned module—if no such similar pre-learned module exists, the parameters are initialized randomly. This transfer learning process [47] is similar to the notion of episodic RL which transfers action/state values from the episodic memory as the agent goes through the learning process [26].

We apply a graph representation to formalize the task modularization and composition processes, although it may take various forms, from a fixed compositional structure to a weighted ensemble or a general function-composition tree [40]. Let $\mathcal{T}$ be a set of related modular tasks with non-parametric variations. Each individual task $t \in \mathcal{T}$ can thus be recast as a graph tuple:

$$G_t = (V, E_t), \quad \forall t \tag{1}$$

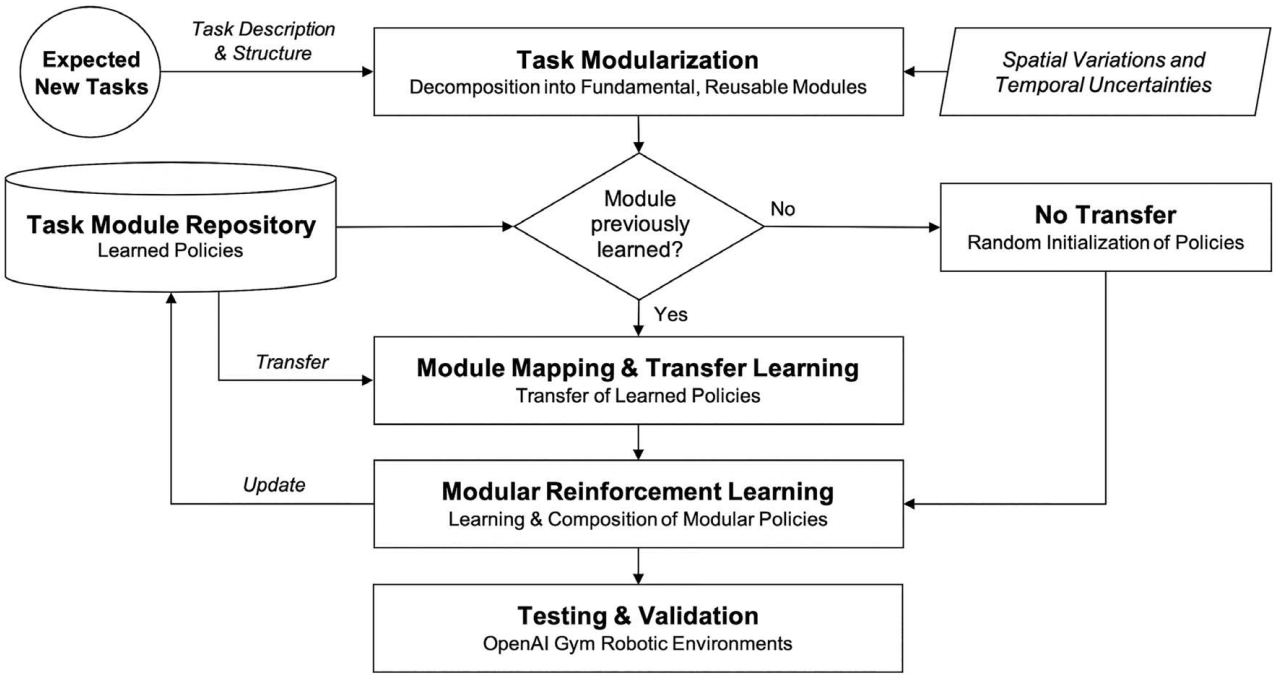


Fig. 2 The modular RL framework

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwxCuIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjYBnBYSI2XgjPCmcDIhbrwgIcQp2IIPRKnjoFGTr58PMjg by Universita Di Pisa user on 27 November 2

where V (i.e., vertices) represent the set of all task modules across $\mathcal{T}$, and $(u, v) \in E_t$ only if u and v ($u \neq v$, $u, v \in V$) are modules of task t, in that sequence. In other words, G_t is a directed graph where the direction of the edges represent the pairwise sequences of the modules constituting task t. The idea of the proposed M-DDPG is thus to learn a separate neural network for each module in V which may be reused when facing new tasks with overlapping modules. That is, $V = \{(F_j, \Theta_j)\}_{j=1\ldots|V|}$, where F_j is the neural network (e.g., DDPG) for module $j \in V$, and Θ_j is the respective set of network parameters. Upon learning task t, this architecture allows some of the modules trained on task t to be reused to "jumpstart" the learning of a new task t' if $E_t \cap E_{t'} \neq \varnothing$.

4.2 Training Module Neural Networks. A DDPG setup with fully observed environments [49] is considered for each task module. Let f and θ denote the set of neural networks and the respective parameters representing a task module. f comprises four neural networks including the actor's policy network, the critic's Q network, the actor's target policy network, and the critic's target Q network, with network parameters denoted by $\theta = (\theta^{\mu}, \theta^{Q}, \theta^{\mu'}, \theta^{Q'})$, in that order. For each given module of a new task, the actor and critic networks are initialized with the parameters of the most related source module denoted by θ^{μ}_{*} and θ^{Q}_{*}. If no such pre-learned module exists, both sets of parameters are initialized randomly. The critic's loss function is calculated as follows:

$$L = 1/N \sum_{i} [y_i - Q(s_i, a_i|\theta^{Q})]^2 \tag{2}$$

where s_i and a_i are, respectively, the agent's state and action at timestep i, $Q(s_i, a_i|\theta^{Q})$ is the state-action pair value given network parameters θ^{Q}, $\theta^{Q}_{init} \leftarrow \theta^{Q}_{*}$, N is the size of the experiences mini-batch, sampled from the replay buffer, and γ is a discount factor. y_i is the Bellman equation [50] calculating the updated Q value as follows:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'}) \tag{3}$$

where r_i is a scalar reward at timestep i, Q' is the target value network with parameters $\theta^{Q'}$, and μ' is the target policy network with parameters $\theta^{\mu'}$. The objective of the policy network is to maximize the expected return $J(\theta) = \mathbb{E}[Q(s, a)|_{s=s_i, a=\mu(s_i)}]$. Since the policy is updated in an *off-policy* fashion, the actor's policy parameters over minibatches of size N is calculated as follows:

$$\nabla_{\theta^{\mu}} J(\theta) = 1/N \sum_{i} [\nabla_a Q(s, a|\theta^{Q})|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu})|_{s=s_i}] \tag{4}$$

where $\theta^{\mu}_{init} \leftarrow \theta^{\mu}_{*}$.

In order to increase governance stability, the notion of soft updates [33,49] is applied for updating the actor and critic target networks as $\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$ and $\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$, respectively ($\tau << 1$). This is accompanied through one-the-fly explorations based on the Ornstein–Uhlenbeck process [89]. The calculated state (s_i) and reward (r_i) together with the actions (a_i) are then utilized to accumulate a replay buffer storing sampled experiences as $\langle s_i, a_i, r_i, s_{i+1} \rangle$ in a finite-size memory for updating neural network parameters. In standard RL, reward is typically assigned from -1, 0 implying missing and achieving a goal g, respectively. We deploy the Hindsight Experience Replay (HER) algorithm [66] which avoids this limitation by rewarding the agent for achieving any intermediate goal g'. For example, g' in the context of robotic material handling would imply reaching a nearby storage bin. The episodes are therefore reexamined with a number of intermediate goals resulting in a replay buffer associated with the original and the intermediate goals. The replay buffer is used for sampling minibatches to update the actor and critic networks.

Algorithm 1 M-DDPG.

Data: Set of tasks $\mathcal{T}$, modular task representations G_t, $\forall t \in \mathcal{T}$, task module repository $\mathcal{R}$
Result: Learn task $t \in \mathcal{T}$ via transfer from module repository $\mathcal{R}$
for *all modules of task t* **do**
  if *there exist a most related pre-trained module in modular repository* $\mathcal{R}$ *with parameters* $\theta = (\theta^{\mu}_{*}, \theta^{Q}_{*}, \theta^{\mu'}_{*}, \theta^{Q'}_{*})$ **then**
    Initialize the critic network $(s, a|\theta^{Q}_{*})$ and actor network $\mu(s|\theta^{\mu}_{*})$ with weight
    $\theta^{\mu}_{*}, \theta^{Q}_{*}$ from θ
  else
    Randomly initialize critic network $(s, a|\theta^{Q})$ and actor network $\mu(s|\theta^{\mu})$ with weight θ^{μ}, θ^{Q}
  end
end
Initialize replay buffer RB and target network Q' and μ' with weights

$$\theta^{\mu'}, \theta^{Q'} \leftarrow \theta^{\mu}, \theta^{Q}$$

for *episode* $= 1 : M$ **do**
  Initialize random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for $i = 1 : T$ **do**
    Select action $a_i = \mu(s|\theta^{\mu}) + \mathcal{N}_i$
    Execute a_i and observe reward r_i and observe new state s_{i+1}
    Store transition (s_i, a_i, r_i, s_{i+1}) in RB
    Sample a random mini-batch of N transitions $(s_i, a_i, r_i, s_{i+1}$ from RB
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update the critic by minimizing the loss:
    $L = 1/N \sum_i [y_i - Q(s_i, a_i|\theta^{Q})]^2$
    Update the actor policy by sampled policy gradient:

$$\nabla_{\theta^{\mu}} J(\theta) = 1/N \sum_{i} [\nabla_a Q(s, a|\theta^{Q})|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu})|_{s=s_i}]$$

    Update the target networks:

$$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$$

$$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$$

  end
end
Add parameters $\theta = (\theta^{\mu}, \theta^{Q}, \theta^{\mu'}, \theta^{Q'})$ of new module to $\mathcal{R}$

5 Experiments

This section evaluates the performance of the proposed framework on a set of OpenAI Gym Robotics environments [48]. OpenAI Gym provides various virtual standard benchmark environments for RL research and has contributed significantly to recent advances in AI research, as evident by the high percentage of publications that use such environments for validation and benchmarking. The existing Robotics environments allow to experiment on a set of virtual goal-oriented tasks including reaching an object positioned randomly in 3D space, pushing/sliding an object to a goal position in 2D space, and picking an object and placing it on a random position in 3D space. The Gym environments are all enabled via the MuJoCo physics simulator [90]. Although the Robotic environments of OpenAI Gym are relatively simplistic versions of real-world collaborative robotic tasks on the shop floor, they offer a powerful platform for building, testing, and validating RL algorithms prior to physical implementation on real cobots. Further, it must be noted that OpenAI Gym provides the capability to create custom-made environments tailored to the details of a real cobot's workspace and operations. For the sake of our experiments and demonstrations, we have used the default Robotics environments of OpenAI Gym, as described in this section.

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=BZN-hBwXcJIAAAAA:svYBgFqmdUMrUaCPqtkZvYAWjYBnBYSI2XgjPCmcDIhbrwgIcQp2IIPRKnjoFGT58PMjg by Universita Di Pisa user on 27 November 2

5.1 Design of Experiments. Each module neural network is trained using DDPG [49] with HER [66] for reward shaping (see Algorithm 1). All four deep neural networks (i.e., training actor network, target actor network, training critic network, and target critic network) comprise three fullyconnected hidden layers with 256 neurons in each layer. Each training epoch consists of 50 cycles with two rollouts per cycle. In each rollout, the cobot interacts with the environment for 50 steps. The network updates the parameters 40 times after 50 interaction steps during each rollout. Other hyperparameters settings are similar to Ref. [66]. For training the M-DDPG algorithm, we first pre-train the networks on a set of tasks and then utilize the pre-trained networks for initializing the similar modules of a new task, as described in Sec. 4. We then allow the agent to interact with the new environment and update the network parameters. The experiments were executed on a CPU AMD Ryzen Threadripper 2970WX 24-Core Processor 3.00GHZ, on which each epoch needed an average of 70 s to run. The experiments are designed to compare the performance of M-DDPG against DDPG as a baseline with respect to jumpstart and asymptotic performance, as suggested by [47].

5.2 Results and Analyses. The proposed framework was implemented and tested on three Robotics environments of the OpenAI Gym. To show the benefit of task modularity, the modules were pre-selected and the process was conducted by decomposing the initial tasks into sub-tasks associated with before and after reaching the object (see Fig. 3). Note that we would want the boundaries of the modules to ultimately be determined as a part of the process. *Learning* the modules boundaries is a critical question that we leave as a future research direction and offer a few potential approaches to tackle this question in the conclusion section.

Consider, for example, Reach and Push as the two source tasks already learned, and Pick-and-Place as the target task to be learned. Now let us define three source task modules: (A) Reach, (B) Push before reaching the object, and (C) Push after reaching the object. Let us also define two target task modules: (D) Pick-and-Place before reaching the object, (E) Pick-and-Place after picking the object up. A potential mapping for this scenario would therefore be D → B or D → A. The experiments have been conducted on two scenarios:

- *Scenario 1* (Fig. 4(*a*)). Transfer the reaching module of the Pick-and-Place task from (i) the Reach task or (ii) the reaching module of the Push task. *Baseline*: DDPG (no transfer).
- *Scenario 2* (Fig. 4(*b*)). Transfer the reaching module of the Push task from (i) the Reach task or (ii) the reaching module of the Pick-and-Place task. *Baseline*: DDPG (no transfer).

We evaluated the performance of the M-DDPG agent against two metrics: success rate and time-to-completion. Although we expect these two metrics to be correlated (as discussed later), it is instructive to keep track of both, since the relative importance of each depends on the application. First, we define a *trail* as a sequence of interactions between the robot and the environment. At the beginning of each trail, the initial parameters of the environment (e.g., positions of the robotic arm/object) are initialized and the robot interacts with the environment until the process reaches one of the following terminal conditions: the robot accomplishes the goal or the maximum number of interaction steps (50, in our experiments) is reached. The former scenario is defined as success. Thus, if the robot can successfully complete the task M_c times out of M trails, its success rate is simply defined as $SR = M_c/M$.

The time-to-completion metric is measured as the number of steps taken by the robot before reaching the goal. This metric only involves successful trials of the robot and is an indicator of the robot's *efficiency* once it is able to complete the task. When the policy of the robot is so poor that it is unable to successfully accomplish the task, the length of the trail approaches infinity. Once the policy improves to the level that the robot can perform the task in some occasions, the length of the trail begins decreasing to a finite value until it converges to a minimum.

The results show that the proposed framework significantly outperforms the baseline DDPG in boosting both the jumpstart and asymptotic performance of the robot [47]. That is, the number of iterations to achieve a 60% success rate has been reduced by over 80% in Scenarios 1 and over 70% in Scenarios 2. Further, the learning process converges much faster in both scenarios by M-DDPG compared to the baseline DDPG. Moreover, the robot is able to fully learn the Pick-and-Place task (transfer from both Reach and Push), while the baseline DDPG is unable to advance beyond 70% success rate on average.

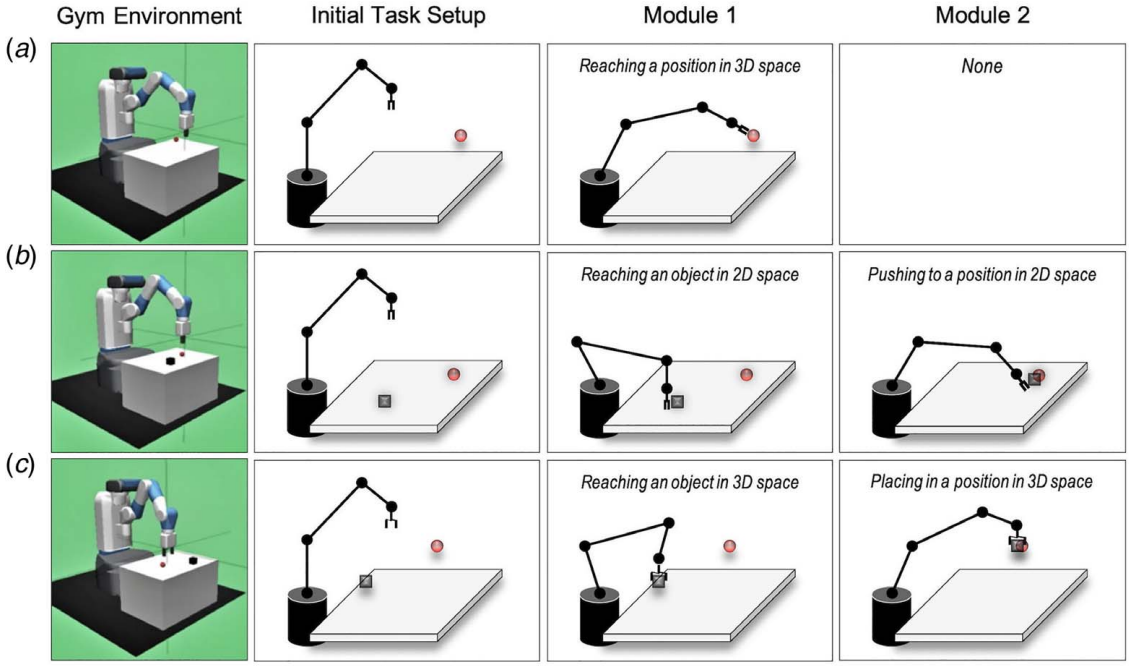


Fig. 3 Experiments setup and task modularization on the OpenAI Gym environments: (*a*) reach a randomly chosen position in 3D space, (*b*) push an object to a randomly chosen position in 2D space (tabletop), and (c) pick-and-place to a randomly chosen position in 3D space

nloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwxCuIAAAAA:svYBgFqmdUMrUaCPqkZvYAWlvBnBYSl2XgjPCmcDIhbrwgkQp2lIPRKnjoFGTr58PMjg by Universita Di Pisa user on 27 November 2

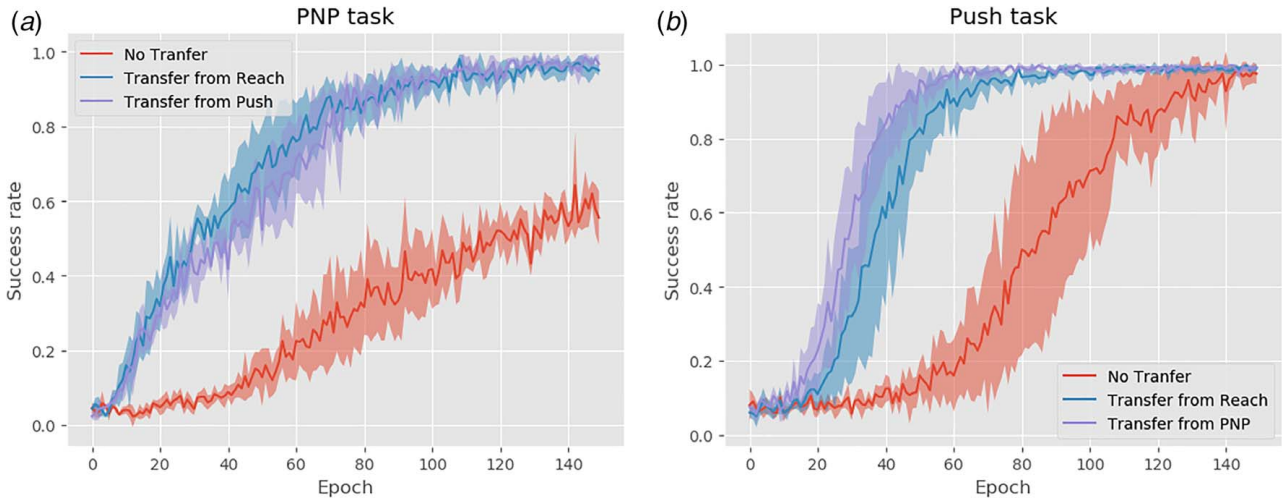


Fig. 4 Success rate under Scenarios (*a*) 1 and (*b*) 2: M-DDPG versus DDPG (no transfer)—significant boost in robot jumpstart and asymptotic performance through transfer learning. Results also indicate the importance of selecting "the right" source task: in Reach, the robot is rewarded for reaching the object regardless of the "side" reached, while in Push and Pick-and-Place, the robot may receive no reward even when it reaches the object if it is on the wrong side.

One interesting observation is that transfer from Reach yields slightly lower performance in Scenario 2, transfer to Push (see Fig. 4). We speculate the underlying reason to be that in Reach, the robot is rewarded for reaching the object regardless of what side of the object it has reached; while in Push the robot may receive no reward even when it reaches the object if it is on the wrong side. This behavior is not much evident in Scenario 1, which may be due to the fact that the Pick-and-Place robot must reach the center of the object not a specific side of it. These observations pose a new question related to transfer across task modules: *If there exists more than one previously learned task modules with slight differences, which one is the best candidate for transfer to the respective module of a new task?* This is followed by two subsequent questions: *Is it possible to transfer from multiple previously learned modules, and if so, how would that affect the robot's performance relative to transfer from only one previously learned module?* Another observation is that the success rate of the robot has a relatively lower variance under M-DDPG. We speculate that this behavior is due to the introduction of an inductive bias [27] associated with transferring from similar, previously learned task modules.

The time-to-completion results (Fig. 5) show a clear advantage for task modularity in terms of this metric: The most efficient policies for performing the two tasks, especially the Pick-and-Place task, require a smaller number of interaction steps between the between and the environment, compared to the cases with no modularity (10 versus 30 for the Pick-and-Place task). Further, the number of iterations to reach a target interaction steps (in this case 35) is reduced significantly (over 80% in Scenario 1 and over 50% in Scenario 2). The success rate and time-to-completion results indicate that the average length of the trail decreases as the robot's success rate improves and finally both metrics converge to an optimum. We assume 90% as a decent level of success rate. In Scenario 1, the robot with no transfer cannot achieve a decent success rate. The robot enhanced with transfer, however, can reach a decent success rate when epoch is 80. The length of trail on average keeps decreasing from 140 epochs to 80 epochs. We can observe a similar behavior in Scenario 2, which suggests that when the robot reaches a decent success rate, it continues to search for a more efficient policy to complete the task faster.

These results also point out the differential benefit of modularity even for such simple tasks with relatively high similarity between

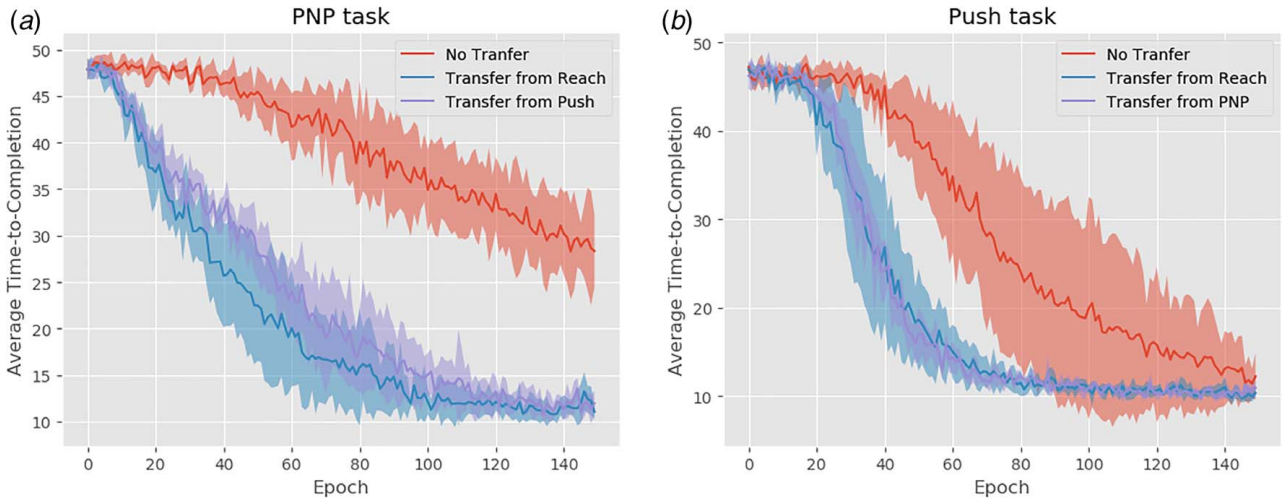


Fig. 5 Time-to-completion (length of the trail) under Scenarios (*a*) 1 and (*b*) 2: M-DDPG versus DDPG (no transfer)—significant improvement in the agent's efficiency in solving through transfer learning.

nloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwXcJIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjYBnBYSI2XgjPCmcDIhbrwgkQp2IIPRKnjoFGTr58PMjg by Universita Di Pisa user on 27 November 2

source and target tasks. As indicated by both metrics, the Pick-and-Place task benefits more from task modularity, at least in our modular decomposition scheme, compared to the Push task. This is in part expected, given the more complex nature of the Pick-and-Place task, and the way the task modularity is pre-selected for these experiments. This is of course only an anecdotal evidence of an important concept and a more formal study of the relative benefits of task modularity requires more precise measures of task complexity and integration of module identification process into the learning algorithms; all crucial topics for future research in this area.

5.3 Discussion: Implications for Industry 4.0 Cobots and Modularity Theory. The results presented in this section signify the feasibility of significantly accelerating the learning process of a cobot through task modularization and transfer of policies across different tasks with non-parametric variations. These findings have two key implications for the Industry 4.0 collaborative robotic applications discussed in Sec. 1:

(1) *Generalization to complex, real-life operations*. Although the practical Industry 4.0 cobot operations such as material handling, machine tending, assembly, and inspection (see Fig. 1) seem to be much more complex than the Reach, Push, and Pick-and-Place tasks studied, their difference becomes less evident at the task module level. That is, the complexity of those practical tasks is primarily a function of the "number" of their constituent modules. Each of those individual modules, however, typically involves simple, fundamental tasks such as reaching an object, pushing/pulling/manipulating it, or transferring it from one location to another. Thus, if the tasks are correctly modularized and the pairwise relationships between the modules are precisely identified, the proposed framework can be easily scaled up and generalized to more complex, real-life Industry 4.0 operations.

(2) *Enabling collaborative learning across fleets of cobots*. The emerging IoT technologies are enabling industrial machines to securely share information with other machines throughout their entire manufacturing ecosystem. With these new possibilities, not only a cobot can accelerate its learning by transferring policies from its own former tasks, it can also exchange policies with tens or hundreds of similar cobots performing similar tasks across the same factory, enterprise, and beyond. Realizing this transformative capability would require rigorous theories and models for task modularization and new metrics to precisely measure the degrees of similarity and transferability between modules with regard to their parent source and target tasks as well as the specifications of their source and target cobots.

It is also instructive to circle back these results to our earlier discussion on modularity and complexity. As stated in Sec. 3, modularity is an effective mechanism to manage complexity, thus, we expect its effectiveness to increase with complexity. This is indeed what we observe here, as the more complex task (pick and place) benefits more from modularity, especially in the long run. Simpler tasks (in this case, push) accrue most of the benefit from the initial jumpstart, although in the long run, benefits of modularity, vis-a-vis a monolithic approach, might fade away. Intuitively, we can consider the problem of learning as a search in a rugged landscape, where the scale of ruggedness depends on the complexity of the problem at hand. In highly rugged landscapes, the advantage of modularity is driven by helping the system to avoid a large number of potential local optima that the learning system can get stuck for a long time on the path to learning. In low complexity landscapes, however, the benefit is largely driven by the proximity of the initial conditions to the global optimum, an advantage that can quickly diminish over time.

To summarize, one should consider the aggregate effect of task complexity and the urgency of learning speed for evaluating the potential benefits of the modular RL framework. As a rule of thumb, we should favor a modular RL approach when either the complexity of the set of target tasks is high, or where there is a fast turn-over of simpler tasks that need to be learned.

6 Conclusions and Future Work

Industry 4.0 systems need to incorporate a high degree of adaptability, flexibility and resilience into current manufacturing systems, primarily by integrating technology automation with machine autonomy. This paper proposes a novel task modularization framework built upon recent advances in transfer and meta-learning to enhance the adaptability of complex and unstructured robotic applications in the context of Industry 4.0. Our research is motivated, on one hand, by the 11.47% compound annual growth in the global market for industrial robotics [91] and an anticipated global market $10.14 billion for cobots by 2025 [11], and on the other hand, by the absence of rigorous methodologies for efficient transfer of tasks in robotics operations for higher autonomy and adaptability, which does not exist in current, start-from-the-scratch RL methods. We incorporate the rich science of complex adaptive systems with recent advances in deep RL to address a fundamental research question: *How can a cobot use task modularity to autonomously adapt to complex and unstructured Industry 4.0 operations and environments through automated transfer of its learned knowledge across several tasks with parametric and/or non-parametric variations?*. We tackled this question through a computational framework for task modularization based on the principles of complex systems theory and module neural network training based on DDPG algorithm. Our proof-of-concept implementation of the proposed M-DDPG on the Robotics environments of the OpenAI Gym indicated significant improvement in both jumpstart and asymptotic performance of the agent compared to the baseline DDPG.

The long-term vision of the authors is to both increase the *range* of robotic operations [92] and minimize reprogramming through task modularization and sharing of the learned knowledge across various tasks and fleets of robots. By integrating the science of complex adaptive systems with AI techniques and scaling the notion of modularization and transfer up to a large network of robots performing a broad range of tasks, the proposed research will result in a continuously evolving, shared knowledge-base that will eventually enable near-real-time learning of new robotic tasks in manufacturing, logistics, and other industrial sectors. This, in turn, will lead to less-labor-intensive, 24/7 operations with significantly shorter lead times, serving the ultimate vision of Industry 4.0 for *lot-size of one* production. Our vision can be further expanded to a longer-term research endeavor to (1) enhance adaptive learning in networks of heterogeneous robots, (2) improve *future of work systems* with human-in-the-loop by enabling transfer learning between humans and piece-picking robots, and (3) extend the developed models and frameworks to other robotic applications beyond manufacturing and logistics.

This paper makes a case for a longer term research direction, the current implementation is at a proof-of-concept stage and has a number of limitations that can be addressed in future studies: First, although the presented experiment provides sufficient evidence for benefits of task modularization, it can be further expanded in complexity in terms of the number of tasks and the diversity of environments to study the impact of these factors on the performance of the proposed method. Moreover, increasing the complexity of experiments brings us to the second limitation of this study, that is the process of identifying the optimal number and composition of task modules, which quickly becomes non-trivial with an increase in complexity. For medium complexity problems, this problem can be addressed by defining a finite set of potential task modularization schemes to choose from, to be used as an input to the learning algorithm during the training stage. The learning algorithm can then determine the scheme that results in the learning

Downloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwxCuIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgIcQp2IIPRKnjoFGT58PMjg by Universita Di Pisa user on 27 November 2

transfer with the highest performance to a set of test environments. However, as the system's complexity increases, identifying the initial set of candidate modularization schemes will become either conceptually challenging (in case of manual selection of schemes), or computationally intractable (in case of considering all possible schemes). In such setups, we expect automating the process of task modularization to be a fundamental future research question in this area. Finally, although simulation-based methods, using software environments such as OpenAI Gym is a predominant validation method for RL research, modular RL algorithms must eventually be lab-tested using cobots on realistic Industry 4.0 operations.

Beyond the Industry 4.0 applications mentioned in this article, modular meta-learning has much wider potentials in the engineering system design. First, given that most engineering systems are designed and operated in dynamic and uncertain environments that vary across applications and evolve over time, incorporating adaptability and resilience into the AI-enabled methods would likely enable many new applications at various stages of systems life cycles. Moreover, this paper is an example of how the engineering system design research can broaden its interaction with the AI community, go beyond *using* AI methods in design, and leverage relevant system-level concepts—in this case, system modularity—to enable some systemic features such as flexibility, adaptability, and resilience into intelligent engineering systems of the future.

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request.

References

[1] Lasi, H., Fettke, P., Kemper, H. G., Feld, T., and Hoffmann, M., 2014, "Industry 4.0," Business Inform. Syst. Eng., **6**(4), pp. 239–242.
[2] Monostori, L., Kádár, B., Bauernhansl, T., Kondoh, S., Kumara, S., Reinhart, G., Sauer, O., Schuh, G., Sihn, W., and Ueda, K., 2016, "Cyber-Physical Systems in Manufacturing," CIRP Ann. - Manufact. Technol., **65**(2), pp. 621–641.
[3] Moghaddam, M., Cadavid, M. N., Kenley, C. R., and Deshmukh, A. V., 2018, "Reference Architectures for Smart Manufacturing: A Critical Review," J. Manuf. Syst., **49**, pp. 215–225.
[4] Luder, A., Schleipen, M., Schmidt, N., Pfrommer, J., and Hensen, R., 2017, "One Step Towards An Industry 4.0 Component," 13th IEEE Conference on Automation Science and Engineering (CASE), Xi'an, China, IEEE, pp. 1268–1273.
[5] Moghaddam, M., and Nof, S. Y., 2017, "The Collaborative Factory of the Future," Int. J. Computer Int. Manufact., **30**(1), pp. 23–43.
[6] Koren, Y., Heisel, U., Jovane, F., Moriwaki, T., Pritschow, G., Ulsoy, G., and Van Brussel, H., 1999, "Reconfigurable Manufacturing Systems," CIRP Ann. - Manufact. Technol., **48**(2), pp. 527–540.
[7] Hofmann, Erik, and Rüsch, Marco, 2017, "Industry 4.0 and the Current Status As Well As Future Prospects on Logistics," Comput. Indus., **89**, pp. 23–34.
[8] Lu, Y., 2017, "Industry 4.0: A Survey on Technologies, Applications and Open Research Issues," J. Indus. Inform. Int., **6**, pp. 1–10.
[9] Zhong, R. Y., Xu, X., Klotz, E., and Newman, S. T., 2017, "Intelligent Manufacturing in the Context of Industry 4.0: A Review," Engineering, **3**(5), pp. 616–630.
[10] Østergaard, E. H., The Role of Cobots in Industry 4.0, 2018.
[11] Collaborative Robots Market Worth $10.1 Billion By 2025 — CAGR: 44.5%, 2019.
[12] Malik, A. A., and Bilberg, A., 2019, "Collaborative Robots in Assembly: A Practical Approach for Tasks Distribution," 52nd CIRP Conference on Manufacturing Systems, Ljubljana, Slovenia, Vol. 81, pp. 665–670.
[13] Dobra, Z., and Dhir, K. S., 2020, "Technology Jump in the Industry: Human–Robot Cooperation in Production," Indus. Robot: Int. J. Rob. Res. Appl., **47**(5), p. 0039.
[14] Bagheri, B., Yang, S., Kao, H.-A., and Lee, J., 2015, "Cyber-Physical Systems Architecture for Self-Aware Machines in Industry 4.0 Environment," IFAC-PapersOnLine, **48**(3), pp. 1622–1627.
[15] Weyer, S., Schmitt, M., Ohmer, M., and Gorecky, D., 2015, "Towards Industry 4.0-Standardization As the Crucial Challenge for Highly Modular, Multi-Vendor Production Systems," Ifac-Papersonline, **48**(3), pp. 579–584.
[16] Hosseini, S., Morshedlou, N., Ivanov, D., Sarder, M. D., Barker, K., and Khaled, A. A., 2019, "Resilient Supplier Selection and Optimal Order Allocation Under Disruption Risks," Int. J. Prod. Econ., **213**, pp. 124–137.
[17] Ivanov, D., and Dolgui, A., 2020, "A Digital Supply Chain Twin for Managing the Disruption Risks and Resilience in the Era of Industry 4.0," Prod. Planning Control, pp. 1–14.
[18] Odonkor, P., and Lewis, K., 2019, "Data-driven Design of Control Strategies for Distributed Energy Systems," ASME J. Mech. Design, **141**(11), p. 111404.
[19] Odonkor, P., and Lewis, K., 2019, "Automated Design of Energy Efficient Control Strategies for Building Clusters Using Reinforcement Learning," ASME J. Mech. Design, **141**(2), p. 021704.
[20] Liao, H., Zhang, W., Dong, X., Poczos, B., Shimada, K., and Kara, L. B., 2020, "A Deep Reinforcement Learning Approach for Global Routing," ASME J. Mech. Des., **142**(6), p. 061701.
[21] Lee, X. Y., Balu, A., Stoecklein, D., Ganapathysubramanian, B., and Sarkar, S., 2019, "A Case Study of Deep Reinforcement Learning for Engineering Design: Application to Microfluidic Devices for Flow Sculpting," ASME J. Mech. Des., **141**(11), p. 111401.
[22] Panchal, J. H., Fuge, M., Liu, Y., Missoum, S., and Tucker, C., 2019, "Special Issue: Machine Learning for Engineering Design," ASME J. Mech. Des., **141**(11), p. 110301.
[23] Wang, J. X., Kurth-Nelson, Z., Tirumala, D., Soyer, H., Leibo, J. Z., Munos, R., Blundell, C., Kumaran, D., and Botvinick, M., "Learning to Reinforcement Learn," pp. 1–17, 2016.
[24] Battaglia, P. W., Hamrick, J. B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gulcehre, C., Song, F., Ballard, A., Gilmer, J., Dahl, G., Vaswani, A., Allen, K., Nash, C., Langston, V., Dyer, C., Heess, N., Wierstra, D., Kohli, P., Botvinick, M., Vinyals, O., Li, Y., and Pascanu, R., 2018, "Relational inductive biases, deep learning, and graph networks," pp. 1–40.
[25] Gupta, A., Mendonca, R., Liu, Y. X., Abbeel, P., and Levine, S., 2018, "Meta-Reinforcement Learning of Structured Exploration Strategies," Adv. Neural Inform Process. Syst., **2018**(ISSUEDecem(NeurIPS)), pp. 5302–5311.
[26] Ritter, S., Wang, J. X., Kurth-Nelson, Z., Jayakumar, S. M., Blundell, C., Pascanu, R., and Botvinick, M., 2018, "Been There, Done That: Meta-Learning With Episodic Recall," 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, Vol. 10, pp. 6929–6938.
[27] Botvinick, M., Ritter, S., Wang, J. X., Kurth-Nelson, Z., Blundell, C., and Hassabis, D., 2019, "Reinforcement Learning, Fast and Slow," Trends Cognit. Sci., **23**(5), pp. 408–422.
[28] Levine, S., Finn, C., Darrell, T., and Abbeel, P., 2016, "End-to-End Training of Deep Visuomotor Policies," J. Mach. Lear. Res..
[29] Duan, Y., Schulman, J., Chen, X., Bartlett, P. L., Sutskever, I., and Abbeel, P., 2017, "RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning," 5th International Conference on Learning Representations, pp. 1–14.
[30] Tamar, A., Thomas, G., Zhang, T., Levine, S., and Abbeel, P., 2017, "Learning From the Hindsight Plan – Episodic MPC Improvement," ICRA 2017 – IEEE International Conference on Robotics and Automation, Marina Bay Sands, Singapore.
[31] Yu, T., Quillen, D., He, Z., Julian, R., Hausman, K., Finn, C., and Levine, S., 2020, "Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning," 4th Annual Conference on Robot Learning (CoRL2020), Cambridge, MA.
[32] Schaul, T., and Schmidhuber, J., 2010, Metalearning.
[33] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, Andreas K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D., 2015, "Human-level Control Through Deep Reinforcement Learning," Nature, **518**(7540), pp. 529–533.
[34] Silver, D., and Hassabis, D., 2017, AlphaGo Zero: Starting From Scratch.
[35] Moravčík, M., Schmid, M., Burch, N., Lisý, V., Morrill, D., Bard, N., Davis, T., Waugh, K., Johanson, M., and Bowling, M., 2017, "DeepStack: Expert-Level Artificial Intelligence in Heads-Up No-Limit Poker," Science, **356**(6337), pp. 508–513.
[36] Gupta, A., Eysenbach, B., Finn, C., and Levine, S., 2018, Unsupervised Meta-Learning for Reinforcement Learning.
[37] Blundell, C., Uria, B., Pritzel, A., Li, Y., Ruderman, A., Leibo, J. Z., Rae, J., Wierstra, D., and Hassabis, D., 2019, "Model-Free Episodic Control," pp. 1–12.
[38] Pritzel, A., Uria, B., Srinivasan, S., Badia, A. P., Vinyals, O., Hassabis, D., Wierstra, D., and Blundell, C., 2017, "Neural Episodic Control," 34th International Conference on Machine Learning, ICML 2017, Stockholm, Sweden, Vol. 6, pp. 4320–4331.
[39] Devin, C., Gupta, A., Darrell, T., Abbeel, P., and Levine, S., 2017, "Learning Modular Neural Network Policies for Multi-Task and Multi-Robot Transfer," ICRA 2017 – IEEE International Conference on Robotics and Automation.
[40] Alet, F., Lozano-Pérez, T., and Kaelbling, L. P., Modular Meta-Learning. *CoRL*, 2018.
[41] Simon, H. A., 1991, "The Architecture of Complexity," *Facets of Systems Science*, Springer US, Boston, MA, pp. 457–476.
[42] Nolfi, S., 1997, "Using Emergent Modularity to Develop Control Systems for Mobile Robots," Adaptive Behav., **5**(3–4), pp. 343–363.
[43] Baldwin, C. Y., and Clark, K. B., 2000, Design rules. Vol. 1, The Power of Modularity, MIT Press.

nloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=bZN-hBwXcJIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjYBnBYSI2XgjPCmcDIhbrwgkQp2IIPRKnjoFGT58PMjg by Universita Di Pisa user on 27 November 2

[44] Sullivan, K. J., Griswold, W. G., Cai, Y., Hallen, B., Sullivan, K. J., Griswold, W. G., Cai, Y., and Hallen, B., 2001, "The Structure and Value of Modularity in Software Design," 8th European Software Engineering Conference held jointly with 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering – ESEC/FSE-9, New York, NY, ACMPress, p. 99.
[45] Gianetto, D. A., and Heydari, B., 2015, "Network Modularity is Essential for Evolution of Cooperation Under Uncertainty," Sci. Rep., **5**(1), p. 9340.
[46] Heydari, B., and Dalili, K., 2015, "Emergence of Modularity in System of Systems: Complex Networks in Heterogeneous Environments," IEEE Syst. J., **9**(1), pp. 223–231.
[47] Taylor, M. E., and Stone, P., 2009, "Transfer Learning for Reinforcement Learning Domains: A Survey," J. Mach. Learning Res., **10**, pp. 1633–1685.
[48] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W., 2016, "OpenAI Gym. arXiv:1606.01540v1, pp. 1–4.
[49] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., Continuous control with deep reinforcement learning. *arXiv:1509.02971 Help — Advanced Search*, Sep. 2016.
[50] Sutton, R. S., and Barto, A. G., 1998, *Reinforcement Learning: An Introduction*, MIT Press, Cambridge, MA.
[51] Bengio, Y., Courville, A., and Vincent, P., 2013, "Representation Learning: A Review and New Perspectives," IEEE Trans. Pattern Anal. Mach. Intell., **35**(8), pp. 1798–1828.
[52] Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D., 2016, "Mastering the Game of Go With Deep Neural Networks and Tree Search," Nature, **529**, pp. 484–489.
[53] Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., Vezhnevets, A. S., Leblond, R., Pohlen, T., Dalibard, V., Budden, D., Sulsky, Y., Molloy, J., Paine, T. L., Gulcehre, C., Wang, Z., Pfaff, T., Wu, Y., Ring, R., Yogatama, D., Wünsch, D., McKinney, K., Smith, O., Schaul, T., Lillicrap, T., Kavukcuoglu, K., Hassabis, D., Apps, C., and Silver, D., 2019, "Grandmaster Level in StarCraft II Using Multi-agent Reinforcement Learning," Nature, **575**, pp. 350–354.
[54] García, J., and Shafie, D., 2020, "Teaching a Humanoid Robot to Walk Faster Through Safe Reinforcement Learning," Eng. Appl. Artificial Intell., **88**, p. 103360.
[55] Vengerov, D., 2007, "A Reinforcement Learning Approach to Dynamic Resource Allocation," Eng. Appl. Artificial Intell., **20**(3), pp. 383–390.
[56] Wang, Y. H., Hseng, T., Li, S., and Lin, C. J., 2013, "Backward Q-learning: The Combination of Sarsa Algorithm and Q-learning," Eng. Appl. Artificial Intell., **26**(9), pp. 2184–2193.
[57] Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y., 2000, "Policy Gradient Methods for Reinforcement Learning With Function Approximation," *Policy Gradient Methods for Reinforcement Learning With Function Approximation*, MIT Press, Cambridge.
[58] Konda, V. R., and Tsitsiklis, J. N., 2000, "Actor-Critic Algorithms," *Actor-Critic Algorithms*, MIT Press, Cambridge.
[59] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K., 2016, "Asynchronous Methods for Deep Reinforcement Learning. 48.
[60] Degris, T., White, M., and Sutton, R. S., 2012, "Off-Policy Actor-Critic," Proceedings of the 29th International Conference on Machine Learning, ICML 2012, Madison, WI.
[61] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., 2017, "Proximal Policy Optimization Algorithms," pp. 1–12.
[62] Bucak, I. O., and Zohdy, M. A., 2001, "Reinforcement Learning Control of Nonlinear Multi-link System," Eng. Appl. Artificial Intell., **14**(5), pp. 563–575.
[63] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M., 2014, "Deterministic Policy Gradient Algorithms," 31st International Conference on Machine Learning, ICML 2014, Beijing, China.
[64] Schaul, T., Quan, J., Antonoglou, I., and Silver, D., 2016, "Prioritized Experience Replay," 4th International Conference on Learning Representations, ICLR 2016 – Conference Track Proceedings, San Juan, Puerto Rico.
[65] Berner, C., Brockman, G., Chan, B., Cheung, V., De˛biak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., de Oliveira Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S., 2019, "Dota 2 With Large Scale Deep Reinforcement Learning.
[66] Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., and Zaremba, W., 2017, "Hindsight Experience Replay," Adv. Neural Inform. Process. Syst., **2017-Dec. (Nips)**, pp. 5049–5059.
[67] Harlow, H. F., The Formation of Learning Sets. *Psychological Review*, 1949.
[68] Moghaddam, M., Chen, Q., and Deshmukh, A. V., 2020, "A Neuro-Inspired Computational Model for Adaptive Fault Diagnosis," Expert Syst. Appl., **140**, p. 112879.
[69] Bengio, Y., 2017, The Consciousness Prior. 1–7.
[70] Mishra, N., Rohaninejad, M., Chen, X., and Abbeel, P., 2018, "A Simple Neural Attentive Meta-learner," 6th International Conference on Learning Representations, ICLR 2018—Conference Track Proceedings, Vancouver, BC, Canada, pp. 1–17.
[71] Simon, H. A., 1962, "The Architecture of Complexity," Proc. Am. Philosophical Soc., **106**(6), pp. 467–482.
[72] Eppinger, S., and Ulrich, K., 2015, *Product Design and Development*, McGraw-Hill Higher Education, New York.
[73] Baldwin, C. Y., and Clark, K. B., 2000, *Design Rules: The Power of Modularity*, Vol. 1, MIT Press, Cambridge.
[74] Moore, W. L., Louviere, J. J., and Verma, R., 1999, "Using Conjoint Analysis to Help Design Product Platforms," J. Product Innovat. Manage., **16**(1), pp. 27–39.
[75] Mosleh, M., and Heydari, B., 2017, "Fair Topologies: Community Structures and Network Hubs Drive Emergence of Fairness Norms," Sci. Rep., **7**(1), p. 2686.
[76] Wixson, L. E., 1991, "Scaling Reinforcement Learning Techniques Via Modularity," *Machine Learning Proceedings*, Elsevier, pp. 368–372.
[77] Uchibe, E., Asada, M., and Hosoda, K., 1996, "Behavior Coordination for a Mobile Robot Using Modular Reinforcement Learning," Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems. IROS'96, Osaka, Japan.
[78] Barto, A. G., and Mahadevan, S., 2003, "Recent Advances in Hierarchical Reinforcement Learning," Discrete Event Dyn. Syst., **13**(1–2), pp. 41–77.
[79] Singh, S. P., 1992, The Efficient Learning of Multiple Task Sequences, MIT Press, Cambridge, pp. 251–258.
[80] Russell, S. J., and Zimdars, A., 2003, "Q-decomposition for Reinforcement Learning Agents," Proceedings of the 20th International Conference on Machine Learning (ICML-03), Menlo Park, CA, pp. 656–663.
[81] Sprague, N., and Ballard, D., 2003, Multiple-Goal Reinforcement Learning with Modular Sarsa.
[82] Simpkins, C., and Isbell, C., 2019, "Composable Modular Reinforcement Learning," Proceedings of the AAAI Conference on Artificial Intelligence, Vancouver, BC, Canada
[83] Frans, K., Ho, J., Chen, X., Abbeel, P., and Schulman, J., 2017, "Meta learning shared hierarchies. arXiv preprint arXiv:1710.09767.
[84] Andreas, J., Rohrbach, M., Darrell, T., and Klein, D., 2016, "Neural Module Networks," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, pp. 39–48.
[85] Chitnis, R., Kaelbling, L. P., and Lozano-Pérez, T., 2019, "Learning Quickly to Plan Quickly Using Modular Meta-learning," International Conference on Robotics and Automation (ICRA), Montreal, Canada, IEEE, pp. 7865–7871.
[86] Devin, C., Gupta, A., Darrell, T., Abbeel, P., and Levine, S., 2017, "Learning Modular Neural Network Policies for Multi-Task and Multi-Robot Transfer," IEEE International Conference on Robotics and Automation (ICRA), Marina Bay Sands, Singapore, IEEE, pp. 2169–2176.
[87] Vecerik, M., Sushkov, O., Barker, D., Rothorl, T., Hester, T., and Scholz, J., 2019, "A Practical Approach to Insertion with Variable Socket Position Using Deep Reinforcement Learning," arXiv:1810.01531v2, pp. 754–760.
[88] Heydari, B., Mosleh, M., and Dalili, K., 2016, From Modular to Distributed Open Architectures: A Unified Decision Framework, May 2016.
[89] Uhlenbeck, G. E., and Ornstein, L. S., 1930, "On the Theory of the Brownian Motion," Phys. Rev., **36**(5), p. 823.
[90] Roboti-LLC. MuJoCo Physics Simulator, 2019.
[91] MHI-Deloitte: MHI Annual Industry Report. Technical report, The Material Handling Institute, 2019.
[92] RightHand_Robotics. Understanding the 3Rs of Robotic Piece-Picking, 2018.

nloaded from http://asmedigitalcollection.asme.org/mechanicaldesign/article-pdf/143/7/071701/6706930/md_143_7_071701.pdf?casa_token=BZN-hBwxCuIAAAAA:svYBgFqmdUMrUaCPqkZvYAWjVBnBYSI2XgjPCmcDIhbrwgkQp2IIPRKnjoFGTr58PMjg by Universita Di Pisa user on 27 November 2